

Lecture 5: Harvard Architecture

Learning Objectives

By the end of this lecture, students will be able to:

- Define Harvard Architecture.
 - Explain the structure of Harvard Architecture.
 - Understand the separate memory concept.
 - Explain how Harvard Architecture differs from Von Neumann Architecture.
 - Understand the advantages and disadvantages of Harvard Architecture.
 - Identify real-world applications of Harvard Architecture.
 - Compare both architectures in terms of performance, complexity, and use cases.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally say:

"In today's Computer Organization and Assembly Language lecture, we'll learn Harvard Architecture, compare it with the Von Neumann Architecture, understand separate instruction and data memory, and see how Harvard Architecture improves CPU performance."

In the previous lecture, we studied the Von Neumann Architecture and learned about the Stored Program Concept, where both instructions and data are stored in the same memory.

We also discussed one of its major limitations, known as the Von Neumann Bottleneck.

Today, we're going to study another important computer architecture called the **Harvard Architecture**.

Unlike the Von Neumann model, Harvard Architecture uses separate memories and separate communication paths for instructions and data. This design allows the CPU to access both at the same time, improving performance in many situations.

By the end of this lecture, you'll clearly understand how Harvard Architecture works, why it was introduced, and how it compares with the Von Neumann Architecture.

So, let's begin.

2. Why Was Harvard Architecture Introduced?

Before understanding Harvard Architecture, let's recall the main problem with the Von Neumann Architecture.

In the Von Neumann model:

- Instructions and data are stored in the **same memory**.
- The CPU uses the **same bus** to access both.

This means the CPU may have to wait when it needs to fetch an instruction and read or write data.

This limitation is called the **Von Neumann Bottleneck**.

To reduce this problem, the **Harvard Architecture** was introduced.

3. What is Harvard Architecture?

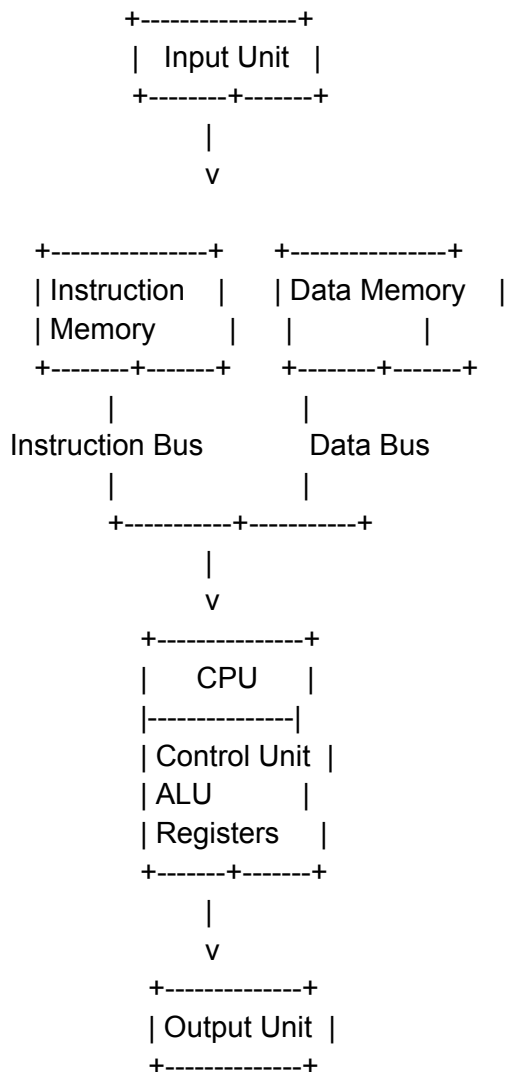
Definition

Harvard Architecture is a computer architecture in which program instructions and data are stored in separate memories and accessed through separate buses.

In simple words:

- One memory stores **instructions**.
 - Another memory stores **data**.
 - The CPU can access both simultaneously.
-

4. Block Diagram of Harvard Architecture



Notice the key difference:

- Separate instruction memory.
- Separate data memory.
- Separate buses.

5. How Harvard Architecture Works

Let's see how a simple program executes.

Suppose we have:

```
int a = 10;  
int b = 20;  
int c = a + b;
```

Step 1

Program instructions are stored in **Instruction Memory**.

Step 2

Variables (**a**, **b**, and **c**) are stored in **Data Memory**.

Step 3

The CPU fetches the next instruction from Instruction Memory.

Step 4

At the same time, it reads the values of **a** and **b** from Data Memory.

Step 5

The ALU performs the addition.

Step 6

The result is written back to Data Memory.

Because instruction fetching and data access use different buses, these operations can occur simultaneously.

6. Memory Organization

In Harvard Architecture, memory is divided into two parts.

MEMORY

+-----+

| Instruction |
| Memory |
+-----+

+-----+
| Data Memory |
+-----+

Instruction Memory

Stores:

- Program instructions
- Machine code

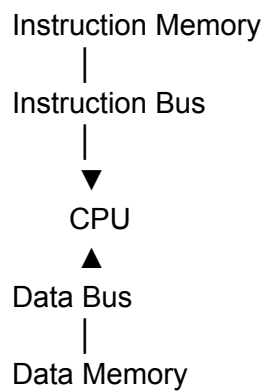
Data Memory

Stores:

- Variables
 - Numbers
 - Arrays
 - Strings
 - Intermediate results
-

7. Separate Buses

One of the biggest advantages is the use of separate buses.



The CPU can:

- Fetch the next instruction.
- Read or write data.

Both operations can happen at the same time.

8. Example of Simultaneous Access

Imagine the CPU needs to:

1. Fetch the instruction:

ADD

2. Read the values:

10
20

In Harvard Architecture:

Instruction Memory



CPU

Data Memory



CPU

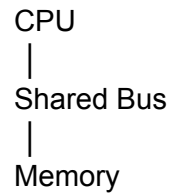
Both accesses occur simultaneously.

This improves performance.

9. Why Is Harvard Architecture Faster?

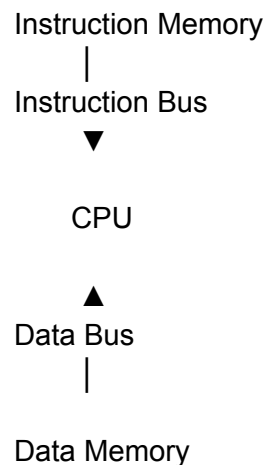
Let's compare.

Von Neumann



The CPU must use the same path for instructions and data.

Harvard



Two independent communication paths allow simultaneous access.

10. Advantages of Harvard Architecture

1. Higher Performance

The CPU can fetch instructions and access data at the same time.

2. Reduced Bottleneck

Separate buses reduce the waiting caused by a shared communication path.

3. Better Throughput

More work can be completed in the same amount of time.

4. Efficient for Embedded Systems

Many embedded systems benefit from this design because they execute predictable instruction streams while processing data efficiently.

5. Supports Parallel Operations

Instruction fetching and data access can proceed in parallel.

11. Disadvantages of Harvard Architecture

1. More Complex Design

Two memories and two buses increase hardware complexity.

2. Higher Cost

Additional hardware generally increases cost.

3. Less Flexible Memory Usage

Instruction memory is intended for instructions, while data memory is intended for data.

Unused space in one cannot necessarily be used for the other without specific architectural support.

4. More Difficult Hardware Design

Managing two separate memory systems requires additional design effort.

12. Real-Life Analogy

Imagine two roads.

Von Neumann

Cars
and
Trucks

↓

One Road

↓

Traffic

Everyone uses the same road.

Traffic becomes slower.

Harvard

Cars

↓

Road 1

↓

Destination

Trucks

↓

Road 2

↓

Destination

Separate roads reduce congestion.

The same idea applies to instruction and data transfers.

13. Harvard vs Von Neumann

This comparison is very important for exams.

Feature	Von Neumann	Harvard
Instruction Memory	Shared	Separate
Data Memory	Shared	Separate
Bus	One	Two
Simultaneous Access	No	Yes
Performance	Lower	Higher
Complexity	Simpler	More Complex
Cost	Lower	Higher

Bottleneck

Present

Greatly
Reduced

14. Practical Example

Suppose the CPU needs to execute:

LOAD A
ADD B
STORE C

In Von Neumann

The CPU:

- Fetches LOAD A.
- Waits.
- Reads A.
- Waits.
- Fetches ADD B.
- Waits.
- Reads B.

Everything competes for the same communication path.

In Harvard

The CPU:

- Fetches the next instruction from Instruction Memory.
- Reads data from Data Memory simultaneously.

This reduces waiting and improves performance.

15. Real-World Applications

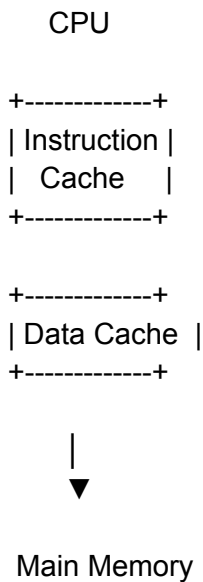
Harvard Architecture is commonly used in:

- Microcontrollers
- Digital Signal Processors (DSPs)
- Embedded systems
- Audio processing devices
- Image processing systems
- Industrial control systems

Many modern processors actually use a **Modified Harvard Architecture**, where separate instruction and data caches are provided even if the main memory is unified.

16. Modified Harvard Architecture

Modern CPUs often combine ideas from both architectures.



This provides many of the performance benefits of Harvard Architecture while keeping a unified main memory.

17. Example: Reading a Program

Suppose your program contains:

```
int sum = a + b;
```

Instruction Memory stores:

```
LOAD A  
LOAD B  
ADD  
STORE SUM
```

Data Memory stores:

```
A = 5  
B = 7  
SUM = 0
```

The CPU fetches instructions while reading data simultaneously, making execution more efficient.

Lecture Summary

Today, we learned about the **Harvard Architecture**, a design that separates program instructions and data into different memories with separate communication paths.

We saw how this allows the CPU to fetch instructions and access data at the same time, reducing the Von Neumann Bottleneck and improving performance.

We also compared Harvard Architecture with Von Neumann Architecture and discussed the advantages, disadvantages, and real-world applications of each.

Finally, we introduced the concept of the **Modified Harvard Architecture**, which combines ideas from both designs and is widely used in modern processors.

In the next lecture, we will begin exploring the **CPU in greater detail**, learning about its internal organization, including the **Control Unit, ALU, Registers, and how they work together to execute instructions**.

Thank you for watching, and I'll see you in the next lecture.